

Hidden Markov Models: Mathematical Notes

Hidden Markov Models (HMMs) are powerful statistical models for modeling sequential data where the underlying process that generates observations is unobservable. They are widely used in domains like speech recognition, natural language processing, and bioinformatics.

An HMM is defined by a set of parameters $\lambda = (A, B, \pi)$:

- **Hidden States (S):** A finite set of N possible hidden states $\{s_1, s_2, \dots, s_N\}$.
- **Observations (O):** A finite set of M possible observation symbols $\{v_1, v_2, \dots, v_M\}$.
- **Initial State Probabilities (π):** $\pi_i = P(q_1 = s_i)$, the probability of starting in state s_i .
- **Transition Probabilities (A):** $A_{ij} = P(q_{t+1} = s_j | q_t = s_i)$, the probability of transitioning from state s_i to state s_j .
- **Emission Probabilities (B):** $B_{jk} = P(o_t = v_k | q_t = s_j)$, the probability of observing v_k when in state s_j .

We will solve two fundamental problems of HMMs using a simple example.

Example Scenario: Daily Activities influenced by Hidden Weather

Let's assume we want to infer the underlying weather (hidden states) based on a person's daily activities (observations).

Model Parameters (λ):

- **Hidden States ($N = 2$):** $S = \{\text{Sunny (S)}, \text{Rainy (R)}\}$
- **Observation Symbols ($M = 3$):** $O = \{\text{Walk (W)}, \text{Shop (H)}, \text{Clean (C)}\}$

1. Initial State Probabilities (π):

$$\pi = \begin{pmatrix} \pi_S \\ \pi_R \end{pmatrix} = \begin{pmatrix} P(q_1 = \text{Sunny}) \\ P(q_1 = \text{Rainy}) \end{pmatrix} = \begin{pmatrix} 0.6 \\ 0.4 \end{pmatrix}$$

2. Transition Probabilities (A):

$$A = \begin{pmatrix} P(S|S) & P(R|S) \\ P(S|R) & P(R|R) \end{pmatrix} = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}$$

(Rows: current state, Columns: next state)

3. Emission Probabilities (B):

$$B = \begin{pmatrix} P(W|S) & P(H|S) & P(C|S) \\ P(W|R) & P(H|R) & P(C|R) \end{pmatrix} = \begin{pmatrix} 0.1 & 0.4 & 0.5 \\ 0.6 & 0.3 & 0.1 \end{pmatrix}$$

(Rows: hidden state, Columns: observation symbol)

Observed Sequence (O): We observe the following sequence of activities over 3 days ($T = 3$):

$$O = (\text{Walk}, \text{Shop}, \text{Clean})$$

Let $o_1 = \text{Walk}$, $o_2 = \text{Shop}$, $o_3 = \text{Clean}$.

I. The Evaluation Problem: Calculating $P(O|\lambda)$ (Forward Algorithm)

Goal: Given the HMM model λ and the observation sequence O , calculate the probability that the model generated this sequence, $P(O|\lambda)$.

Algorithm: The Forward Algorithm (Dynamic Programming)

Let $\alpha_t(i)$ be the probability of the partial observation sequence $o_1, \dots, o_t$ and being in state s_i at time t .

$$\alpha_t(i) = P(o_1, \dots, o_t, q_t = s_i | \lambda)$$

Initialization ($t = 1$): For each state i :

$$\alpha_1(i) = \pi_i \cdot B_{i,o_1}$$

Where B_{i,o_1} is the emission probability of observing o_1 given state s_i . ($o_1 = \text{Walk}$, index for W is 1).

- $\alpha_1(S) = \pi_S \cdot B_{S, \text{Walk}} = 0.6 \cdot 0.1 = 0.06$
- $\alpha_1(R) = \pi_R \cdot B_{R, \text{Walk}} = 0.4 \cdot 0.6 = 0.24$

Recursion ($t = 2, \dots, T$): For each state j :

$$\alpha_t(j) = \left[\sum_{i=1}^N \alpha_{t-1}(i) \cdot A_{ij} \right] \cdot B_{j, o_t}$$

This sums the probabilities of reaching state j at time t from all possible previous states i at time $t-1$, then multiplies by the emission probability of o_t from state j .

For $t = 2$ ($o_2 = \text{Shop}$, index for **H is 2):**

- $\alpha_2(S) = [\alpha_1(S) \cdot A_{SS} + \alpha_1(R) \cdot A_{RS}] \cdot B_{S, \text{Shop}} = [0.06 \cdot 0.7 + 0.24 \cdot 0.4] \cdot 0.4 = [0.042 + 0.096] \cdot 0.4 = 0.138 \cdot 0.4 = 0.0552$
- $\alpha_2(R) = [\alpha_1(S) \cdot A_{SR} + \alpha_1(R) \cdot A_{RR}] \cdot B_{R, \text{Shop}} = [0.06 \cdot 0.3 + 0.24 \cdot 0.6] \cdot 0.3 = [0.018 + 0.144] \cdot 0.3 = 0.162 \cdot 0.3 = 0.0486$

For $t = 3$ ($o_3 = \text{Clean}$, index for **C is 3):**

- $\alpha_3(S) = [\alpha_2(S) \cdot A_{SS} + \alpha_2(R) \cdot A_{RS}] \cdot B_{S, \text{Clean}} = [0.0552 \cdot 0.7 + 0.0486 \cdot 0.4] \cdot 0.5 = [0.03864 + 0.01944] \cdot 0.5 = 0.05808 \cdot 0.5 = 0.02904$
- $\alpha_3(R) = [\alpha_2(S) \cdot A_{SR} + \alpha_2(R) \cdot A_{RR}] \cdot B_{R, \text{Clean}} = [0.0552 \cdot 0.3 + 0.0486 \cdot 0.6] \cdot 0.1 = [0.01656 + 0.02916] \cdot 0.1 = 0.04572 \cdot 0.1 = 0.004572$

Termination: The probability of the observation sequence is the sum of $\alpha_T(i)$ for all states i :

$$P(O|\lambda) = \sum_{i=1}^N \alpha_T(i)$$

$$P(O|\lambda) = \alpha_3(S) + \alpha_3(R) = 0.02904 + 0.004572 = 0.033612$$

Thus, the probability of observing "Walk, Shop, Clean" given our HMM is approximately 0.0336.

II. The Decoding Problem: Finding the Most Likely State Sequence (Viterbi Algorithm)

Goal: Given the HMM model λ and the observation sequence O , find the single best sequence of hidden states $Q^* = (q_1^*, q_2^*, \dots, q_T^*)$ that generated O .

Algorithm: The Viterbi Algorithm (Dynamic Programming)

Let $\delta_t(i)$ be the maximum probability of a path ending in state s_i at time t that produced the partial observation sequence $o_1, \dots, o_t$.

$$\delta_t(i) = \max_{q_1, \dots, q_{t-1}} P(o_1, \dots, o_t, q_1, \dots, q_{t-1}, q_t = s_i | \lambda)$$

To reconstruct the path, we use $\psi_t(i)$, which stores the backpointer (the previous state that maximized $\delta_t(i)$).

Initialization ($t = 1$): For each state i :

$$\delta_1(i) = \pi_i \cdot B_{i, o_1}$$

$$\psi_1(i) = 0 \quad (\text{or a null pointer})$$

- $\delta_1(S) = \pi_S \cdot B_{S, \text{Walk}} = 0.6 \cdot 0.1 = 0.06$
- $\delta_1(R) = \pi_R \cdot B_{R, \text{Walk}} = 0.4 \cdot 0.6 = 0.24$

Recursion ($t = 2, \dots, T$): For each state j :

$$\delta_t(j) = \left[\max_{i=1}^N (\delta_{t-1}(i) \cdot A_{ij}) \right] \cdot B_{j, o_t}$$

$$\psi_t(j) = \arg \max_{i=1}^N (\delta_{t-1}(i) \cdot A_{ij})$$

For $t = 2$ ($o_2 = \text{Shop}$):

- For $\delta_2(S)$: $\delta_1(S) \cdot A_{SS} = 0.06 \cdot 0.7 = 0.042$ $\delta_1(R) \cdot A_{RS} = 0.24 \cdot 0.4 = 0.096$ $\max = 0.096$ (from state R) $\delta_2(S) = 0.096 \cdot B_{S,Shop} = 0.096 \cdot 0.4 = 0.0384$ $\psi_2(S) = R$
- For $\delta_2(R)$: $\delta_1(S) \cdot A_{SR} = 0.06 \cdot 0.3 = 0.018$ $\delta_1(R) \cdot A_{RR} = 0.24 \cdot 0.6 = 0.144$ $\max = 0.144$ (from state R) $\delta_2(R) = 0.144 \cdot B_{R,Shop} = 0.144 \cdot 0.3 = 0.0432$ $\psi_2(R) = R$

For $t = 3$ ($o_3 = \text{Clean}$):

- For $\delta_3(S)$: $\delta_2(S) \cdot A_{SS} = 0.0384 \cdot 0.7 = 0.02688$ $\delta_2(R) \cdot A_{RS} = 0.0432 \cdot 0.4 = 0.01728$ $\max = 0.02688$ (from state S) $\delta_3(S) = 0.02688 \cdot B_{S,Clean} = 0.02688 \cdot 0.5 = 0.01344$ $\psi_3(S) = S$
- For $\delta_3(R)$: $\delta_2(S) \cdot A_{SR} = 0.0384 \cdot 0.3 = 0.01152$ $\delta_2(R) \cdot A_{RR} = 0.0432 \cdot 0.6 = 0.02592$ $\max = 0.02592$ (from state R) $\delta_3(R) = 0.02592 \cdot B_{R,Clean} = 0.02592 \cdot 0.1 = 0.002592$ $\psi_3(R) = R$

Termination: Find the state q_T^* that maximizes $\delta_T(i)$:

$$P^* = \max_{i=1}^N \delta_T(i)$$

$$q_T^* = \arg \max_{i=1}^N \delta_T(i)$$

$$P^* = \max(0.01344, 0.002592) = 0.01344. \quad q_3^* = S \text{ (Sunny)}$$

Path Backtracking: For $t = T - 1$ down to 1:

$$q_t^* = \psi_{t+1}(q_{t+1}^*)$$

- $q_2^* = \psi_3(q_3^*) = \psi_3(S) = S$ (Sunny)
- $q_1^* = \psi_2(q_2^*) = \psi_2(S) = R$ (Rainy)

Most Likely State Sequence: The most likely sequence of hidden states for the observation "Walk, Shop, Clean" is:

$$Q^* = (\text{Rainy}, \text{Sunny}, \text{Sunny})$$

Discussion:

- **Dynamic Programming Efficiency:** Both the Forward and Viterbi algorithms employ dynamic programming to avoid redundant calculations. A naive approach would involve summing/maximizing over all N^T possible state sequences, which is computationally infeasible for large T . Dynamic programming reduces this to $O(N^2T)$ complexity.
- **Log-Probabilities for Numerical Stability:** Similar to Naive Bayes, in practical HMM implementations, probabilities are often converted to log-probabilities (e.g., $\log(\alpha_t(j)) = \log(\sum \dots) + \log(B_{j,o_t})$) to prevent underflow issues that arise from multiplying many small probabilities. This transforms sums into 'logsumexp' operations and products into sums.
- **The Learning Problem (Baum-Welch Algorithm):** We focused on evaluation and decoding. The third fundamental problem, learning the HMM parameters (A, B, π) from observed sequences, is typically solved using the Baum-Welch algorithm. This is an Expectation-Maximization (EM) algorithm:
 - **E-step (Expectation):** Calculate the expected number of transitions and emissions for each state using the current model parameters (involving Forward and Backward algorithm calculations).
 - **M-step (Maximization):** Re-estimate the model parameters based on these expected counts.

This iterative process converges to a local maximum of the likelihood function.

• Limitations of HMMs:

- **Markov Assumption:** The "memoryless" property (future state depends only on the current state) is a strong assumption. Many real-world sequences have longer-range dependencies.
- **Fixed Number of States:** The number of hidden states must be predetermined.

- **HMMs vs. Modern Sequence Models:** While HMMs are foundational, more recent models like Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and Transformers have surpassed HMMs in performance for many complex sequence modeling tasks (e.g., state-of-the-art speech recognition or machine translation). These neural approaches can model longer-range dependencies and learn feature representations directly from raw data. However, HMMs remain valuable for:
 - **Interpretability:** Their probabilistic structure allows for clear interpretation of state transitions and emissions.
 - **Smaller Datasets:** HMMs can perform well with less training data compared to deep neural networks.
 - **Foundational Understanding:** They provide a crucial stepping stone for understanding more complex probabilistic graphical models and sequence models.

Prepared By:

Md. Atikuzzaman

Lecturer

Department of Computer Science and Engineering

Green University of Bangladesh

Email: atik@cse.green.edu.bd